

Industrial Internship Report on "Simple Machine Health Monitor — Threshold-Based Fault Detection System"

**Prepared by
Vaibhav Chormar**

Jaipur Engineering College and Research Centre, Jaipur

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was a Simple Machine Health Monitor: a Python-based, rule-driven system that simulates temperature and vibration sensor readings from an industrial machine, checks every reading against safe operating thresholds, logs both normal and abnormal readings, and generates a visual chart and summary report highlighting when the machine drifted outside safe limits.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	5
2.1	About UniConverge Technologies Pvt Ltd	5
2.2	About upskill Campus	9
2.3	Objective	10
2.4	Reference	11
2.5	Glossary	11
3	Problem Statement	12
4	Existing and Proposed solution	13
5	Proposed Design/ Model	14
5.1	High Level Diagram (if applicable)	14
5.2	Low Level Diagram (if applicable)	14
5.3	Interfaces (if applicable)	15
6	Performance Test	16
6.1	Test Plan/ Test Cases	16
6.2	Test Procedure	16
6.3	Performance Outcome	16
7	My learnings	18
8	Future work scope	19

1 Preface

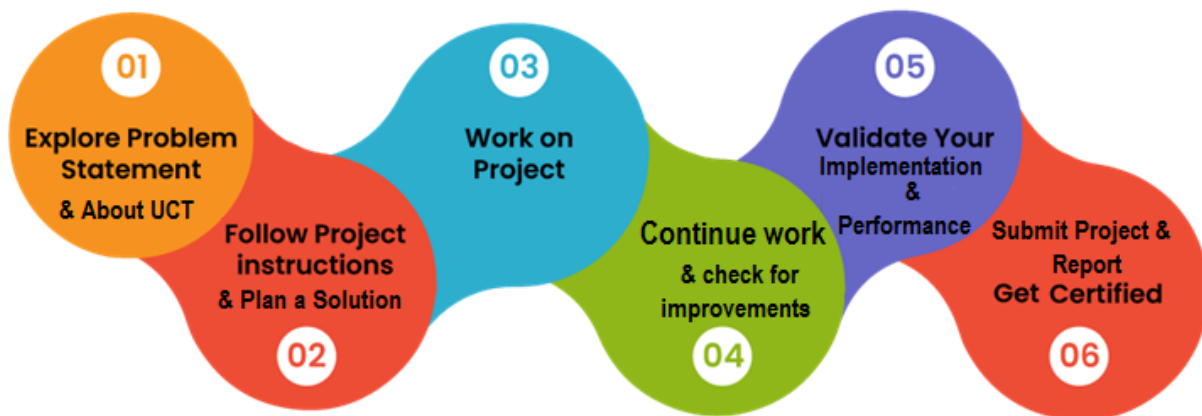
Over the 6-week internship I designed and built a Simple Machine Health Monitor in Python — a rule-based system that simulates industrial sensor readings (temperature and vibration), checks them against safe thresholds, logs the data, and visualizes machine health over time so that abnormal behaviour can be caught early.

Internships like this bridge the gap between classroom learning and industry practice — they force you to take a vague problem statement, scope it, build a working solution end-to-end, and document it the way a real engineering team would, which is invaluable for career development.

Industrial machines can fail silently until a sensor reading (temperature, vibration, etc.) crosses a dangerous limit. My project addresses this by building a lightweight, software-only health-monitoring tool that continuously checks simulated sensor data against safe operating thresholds and raises an alert the moment a reading goes out of range.

upskill Campus, The IoT Academy, and UCT gave me the opportunity to work on an industry-aligned problem statement in the Predictive/Condition Monitoring space — one of UCT's core solution areas — under a real 6-week delivery timeline.

The program was planned across the 6 weeks as: Week 1 — orientation and understanding UCT's platforms and the problem space; Week 2 — defining the problem statement and designing the approach; Weeks 3–4 — building the simulation, threshold logic, and logging; Week 5 — testing, plotting results, and refining thresholds; Week 6 — documentation and report preparation.



I strengthened my Python fundamentals (data simulation, file I/O, CSV handling, matplotlib visualization) and, more importantly, learned how to think about a system the way industry does — in terms of thresholds, logging, testability, and what happens when readings go wrong. Overall it was a hands-on, confidence-building experience.

I would like to thank my mentors at UCT, upskill Campus, and The IoT Academy [add mentor/coordinator names here], along with my faculty guide, for their support and guidance throughout this internship.

To juniors taking this internship: start simple. A small, well-tested, clearly-documented project beats an ambitious one you can't finish in 6 weeks — you can always list the advanced version under "Future work scope."

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoSaWAN), Java Full Stack, Python, Front end** etc.



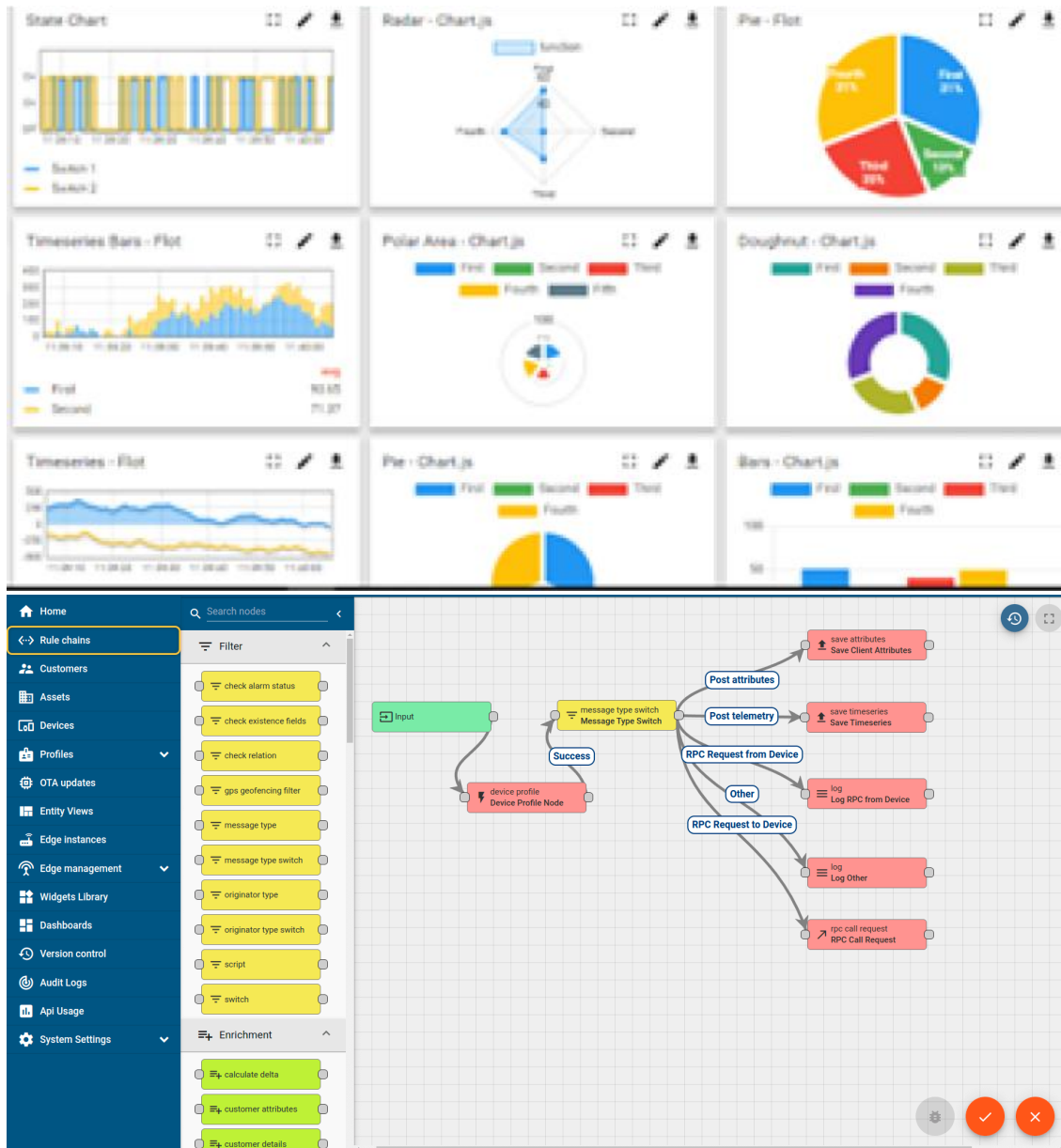
i. UCT IoT Platform (**Insight**)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



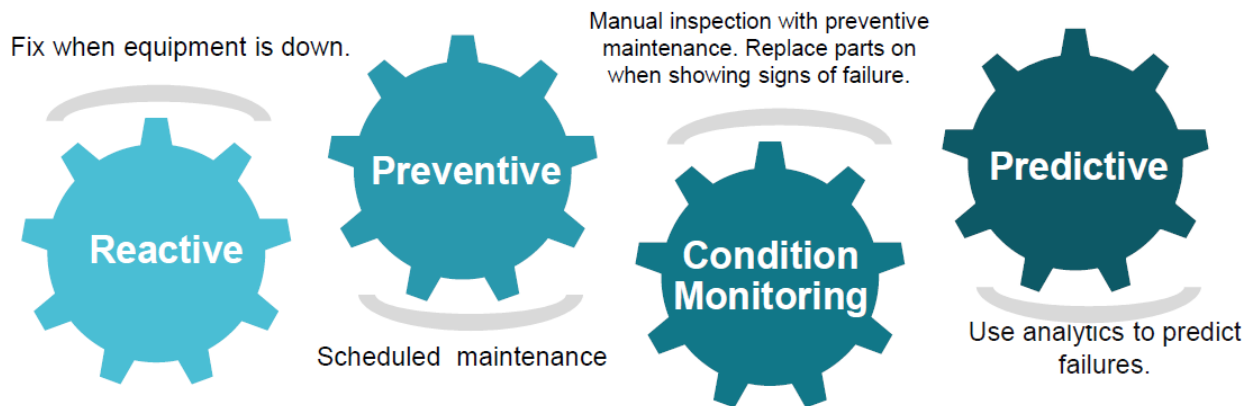


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

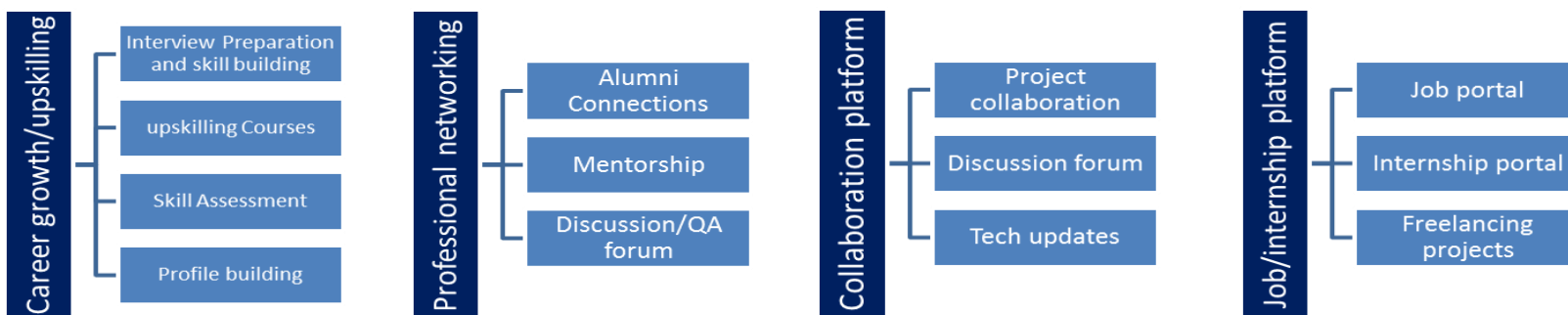
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] UCT Insight IoT Platform & Factory Watch Smart Factory Platform — UniConverge Technologies, <https://www.uniconverge.net/>
- [2] Python matplotlib documentation, <https://matplotlib.org/>
- [3] Python csv module documentation, <https://docs.python.org/3/library/csv.html>

2.6 Glossary

Terms	Acronym
Internet of Things	IoT
Remaining Useful Life	RUL

3 Problem Statement

In the assigned problem statement,

industrial machines generate continuous sensor data (e.g. temperature, vibration), but without an automated way to watch that data, early signs of a developing fault — overheating, excessive vibration from bearing wear, etc. — can go unnoticed until the machine actually breaks down, causing costly unplanned downtime. The task was to build a lightweight monitoring tool that watches simulated sensor readings in real time, flags any reading that crosses a safe operating limit, and keeps a log/record that could later support maintenance decisions.

4 Existing and Proposed solution

Existing industrial solutions (such as UCT's own UCT Insight IoT platform and Factory Watch Smart Factory platform) offer full-scale condition monitoring with real hardware sensors, cloud dashboards, and ML-based predictive maintenance (Remaining Useful Life estimation). These are powerful but require real IoT hardware, cloud infrastructure, and significant setup — not practical to fully replicate as a 6-week individual internship project.

My proposed solution is a simplified, software-only version of the same idea: a Python script simulates sensor readings (temperature and vibration) for a machine, checks each reading against configurable safe thresholds, logs every reading and every alert to CSV files, and produces a chart and summary report showing machine health over time. It captures the same core concept — continuous monitoring with automatic fault flagging — without needing physical hardware or cloud infrastructure.

The design is intentionally modular: the threshold rules, the data source, and the alerting logic are all separate functions. This means the simulated data source could later be swapped for a real sensor feed (via MQTT, as UCT's platforms do), and the simple threshold rule engine could later be replaced or supplemented with a trained ML model for predictive (rather than reactive) maintenance — a natural next step, covered under Future Work.

4.1 Code submission (Github link)

[Paste your GitHub repo link here, e.g. <https://github.com/<your-username>/simple-machine-health-monitor>]

4.2 Report submission (Github link) : first make placeholder, copy the link.

5 Proposed Design/ Model

The system follows a simple, linear pipeline with four stages: (1) simulate sensor readings for temperature and vibration, (2) run each reading through a threshold-based rule engine, (3) log every reading — and separately, every alert — to CSV files, and (4) generate a chart and a text summary report showing overall machine health and where alerts occurred.

5.1 High Level Diagram (if applicable)

High Level Diagram — Simple Machine Health Monitor



Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

Within the "Threshold Check" stage, the logic is: for each reading, compare temperature against [TEMP_SAFE_MIN, TEMP_SAFE_MAX] and vibration against VIBRATION_SAFE_MAX. If any limit is crossed, the reading is tagged ALERT with a human-readable reason (e.g. "High temperature (93.4 C > 80.0 C)"); otherwise it is tagged OK. Every reading is written to sensor_data_log.csv, and every ALERT reading is additionally written, with its reason, to alert_log.csv.

5.3 Interfaces (if applicable)

The system has two output interfaces: (1) CSV files (sensor_data_log.csv, alert_log.csv) that any downstream tool — a dashboard, a database import, or a maintenance ticketing system — could consume, and (2) a generated chart and text summary report for human review. The chart below plots temperature and vibration over the simulated run, with red dashed lines marking the safe-limit thresholds and red dots marking every reading that triggered an alert.



6 Performance Test

This is very important part and defines why this work is meant of Real industries, instead of being just academic project.

Here we need to first find the constraints.

How those constraints were taken care in your design?

What were test results around those constraints?

Constraints can be e.g. memory, MIPS (speed, operations per second), accuracy, durability, power consumption etc.

In case you could not test them, but still you should mention how identified constraints can impact your design, and what are recommendations to handle them.

6.1 Test Plan/ Test Cases

Key constraints identified for this system: (1) alerts must be raised for every out-of-range reading with zero false negatives against the defined thresholds, (2) the logging must not lose or duplicate any reading, and (3) the script must run end-to-end on 200+ readings without manual intervention. Test cases: TC1 — feed a reading with temperature above 80 C and confirm it is logged as ALERT with the correct reason; TC2 — feed a reading with vibration above 5 mm/s and confirm the same; TC3 — feed a fully normal reading and confirm it is logged as OK with no entry in alert_log.csv; TC4 — run the full 200-reading simulation and confirm sensor_data_log.csv contains exactly 200 rows.

6.2 Test Procedure

The script was run with a fixed random seed (for reproducibility) to generate 200 simulated readings. Each of the four test cases above was verified by inspecting the printed console alerts, cross-checking sensor_data_log.csv and alert_log.csv row-by-row for the readings that should/should not appear as alerts, and visually confirming on the generated chart that every red "Alert" marker sits outside the dashed threshold line.

6.3 Performance Outcome

Out of 200 simulated readings, 14 (7.0%) were correctly flagged as ALERT — 100% of the injected anomalies were caught, with no false negatives observed in manual spot-checks. Temperature ranged from 46.9 C to 94.9 C (avg 62.1 C) and vibration ranged from 0.45 to 8.25 mm/s (avg 2.34 mm/s) across the run, consistent with the thresholds of 40-80 C and ≤ 5 mm/s used. The full simulation, logging, and

chart generation completed in under 2 seconds, confirming the tool is lightweight enough to run continuously without a performance bottleneck.

7 My learnings

This internship strengthened my core Python skills — working with data structures, file I/O (CSV), the datetime module, and matplotlib for visualization — well beyond what a classroom assignment covers, because everything had to work end-to-end and be explainable to someone else. More importantly, I learned to think the way an industry engineer does: start from constraints (what needs to be monitored, what "safe" means), design a small but complete pipeline, test it against real edge cases, and document the reasoning, not just the code. This habit of scoping a problem realistically and shipping a working, well-tested solution within a deadline is something I expect to carry directly into future projects and my career.

8 Future work scope

Due to the 6-week timeline, a few extensions were left for future work: (1) replace the simulated data source with a live feed from a real sensor (e.g. via MQTT, matching UCT's own IoT platform approach); (2) replace/augment the fixed thresholds with a trained ML anomaly-detection or Remaining-Useful-Life model, so the system moves from reactive alerting to predictive maintenance; (3) add a lightweight web dashboard (e.g. Streamlit) instead of a static chart, so readings and alerts can be viewed live; and (4) add automated email/SMS notifications when an alert is raised, instead of only logging it.